

Lesson 4: useEffect and Fetching Live Data in React

Mission Brief

Mission Control needs a live feed from ground stations — but fetching data is a "side effect," something that reaches outside your component to talk to the network, and doing it at the wrong moment causes infinite loops or requests that fire on every keystroke. Today you learn `useEffect`, the hook React gives you to run code in sync with a component's lifecycle, and use it to fetch real data from an API when a component first appears.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain what a "side effect" is and why it needs special handling in React
- Use `useEffect` to run code when a component mounts
- Fetch data from an API inside `useEffect` and store it in state
- Handle loading and error states while data is in flight
- Use the dependency array to control when an effect re-runs

What You'll Build

A `MissionFeed` component that fetches a list of "missions" from a public API on load, shows a loading indicator while waiting, displays the results in a list once they arrive, and shows an error message if the request fails.

Prerequisites

- Comfortable with `useState` and event handlers (Lesson 3)
- Basic understanding of `fetch` and Promises (Course 08, Lessons 4-5)

Key Concepts

- Side effects: anything outside of "compute and return UI"
- `useEffect(fn, deps)` and the dependency array
- Fetching on mount with an empty dependency array `[]`
- Loading / success / error state pattern
- Cleanup functions (conceptual introduction)

Concept Explanation

Rendering a component should be a pure calculation: given the same props and state, it returns the same JSX, with no side effects like network calls, timers, or manually touching the DOM. But real apps *need* side effects — fetching data being the most common one. React's `useEffect` hook is the sanctioned place to put them.

```
import { useState, useEffect } from "react";

function MissionFeed() {
  const [missions, setMissions] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch("https://api.example.com/missions")
      .then((res) => {
        if (!res.ok) throw new Error("Request failed: " + res.status);
        return res.json();
      })
      .then((data) => setMissions(data))
      .catch((err) => setError(err.message))
      .finally(() => setLoading(false));
  }, []);

  if (loading) return <p>Loading missions...</p>;
  if (error) return <p>⚠️ {error}</p>;

  return (
    <ul>
      {missions.map((m) => (
        <li key={m.id}>{m.name}</li>
      ))}
    </ul>
  );
}
```

`useEffect` takes two arguments: a function to run, and a **dependency array** that controls *when* it runs. An empty array `[]` means "run this function exactly once, right after the component's first render, and never again" — perfect for a one-time fetch on load. If you omitted the array entirely, the effect would re-run after every render, which for a fetch would mean hammering the API in an infinite loop, since each fetch eventually calls a setter, which triggers a re-render, which re-runs the effect, forever. If you passed `[someValue]`, the effect re-runs only when `someValue` changes between renders — useful for "re-fetch when the search term changes" style features.

The loading/error/success pattern shown above is the standard shape for any data-fetching component: three state variables, an effect that eventually settles into either success or error, and conditional rendering based on which state you're in. Always initialize `loading` to `true` (there's a fetch in flight from the very first render) and always handle the possibility that `fetch` throws or that the response isn't `ok` — networks fail, and a component that silently shows nothing on failure is a broken user experience.

`useEffect` functions can also return a **cleanup function**, which React calls before the effect runs again or when the component is removed from the page — commonly used to cancel a timer or an in-flight request. You'll use this more deeply in later, more advanced work; for now, know it exists:

```
useEffect(() => {
  const id = setInterval(() => console.log("tick"), 1000);
  return () => clearInterval(id); // cleanup: stop the interval
}, []);
```

Guided Walkthrough

1. Create `src/components/MissionFeed.jsx` with the three state variables (`missions`, `loading`, `error`) and the `useEffect` fetch shown above, but point it at a real public API such as `https://jsonplaceholder.typicode.com/posts?_limit=8` for testing (map its `title` field instead of `name`).
2. Render `<MissionFeed />` in `App.jsx` and confirm you briefly see "Loading missions..." before the list appears.
3. Temporarily change the URL to an invalid one (like `https://jsonplaceholder.typicode.com/does-not-exist`) and confirm your error message renders instead of a blank screen or a crash.
4. Add a `refreshKey` state number and a "Refresh" button that increments it; add `refreshKey` to the effect's dependency array so clicking Refresh re-fetches the list, and reset `loading` to `true` at the start of the effect so the loading message reappears each time.
5. Open your browser's Network tab while clicking Refresh a few times to confirm exactly one request fires per click — not more.

Common Bugs & Debugging Tips

- Omitting the dependency array entirely causes an infinite fetch loop — the Network tab will show requests firing continuously; always pass at least `[]`.
- Forgetting `.catch()` means a network failure leaves `loading` stuck at `true` forever, with no error shown — always pair a `.catch` with a `.finally(() => setLoading(false))` or equivalent.
- Trying to `await fetch(...)` directly inside the function passed to `useEffect` doesn't work because that function cannot be `async` itself — instead define an inner `async` function `load() { ... }` and call `load()`, or use `.then()` chains as shown above.
- Rendering `missions.map(...)` before the data has arrived, without a loading guard, throws or renders nothing useful — always guard with `if (loading) return ...` first.

Check Your Understanding

1. What problem occurs if you omit the dependency array from `useEffect`?

2. Why should `loading` start out `true` rather than `false` ?
3. What does passing `[]` as the dependency array mean?
4. Why can't the function passed directly to `useEffect` be declared `async` ?
5. What is a cleanup function and when does React call it?

Mini Quiz + Answer Key

Q1. What does `useEffect(fn, [])` mean? A) Run `fn` on every render B) Never run `fn` C) Run `fn` once, after the first render D) Run `fn` before the component renders

Q2. What happens if you leave out the dependency array completely? A) Nothing changes B) The effect runs after every render, which can loop forever if it sets state C) The effect runs once D) It causes a syntax error

Q3. Why do we track a separate `error` state instead of just letting a failed fetch silently do nothing? A) It's required by JavaScript B) So the UI can show the user something went wrong instead of an unexplained blank screen C) It makes fetch faster D) It prevents re-renders

Q4. What is the correct way to add `async` code inside `useEffect` ? A) Make the effect function itself `async` B) Define and call an inner `async` function, or use `.then()` chains C) It's not possible D) Use `await` at the top level of the component

Answer Key: 1-C, 2-B, 3-B, 4-B

Lesson Recap

You learned that side effects like network requests belong inside `useEffect`, that the dependency array controls when an effect re-runs (with `[]` meaning "once, on mount"), and how to build the standard loading/error/success pattern for any data-fetching component.

Homework

Build a `PlanetWeather` component that fetches from `https://jsonplaceholder.typicode.com/users` (pretend each "user" is a planet outpost), shows a loading state, displays each entry's `name` and `email` in a card once loaded, includes a working error path (test it with a bad URL), and includes a "Reload" button using the `refreshKey` pattern from the walkthrough.

Portfolio Project Connection

Project 08's social feed and Project 09's projects list both fetch or load data on mount using this exact loading/error/success `useEffect` pattern before rendering real content to the page.